

Final Project

Sean Niez

May 2026

Abstract

In practice, linear regression models are fit to a finite sample that is assumed to be representative of the underlying system they attempt to model. However, it is often the case that a sample may have too few data points to be considered representative, or perhaps that the data-generating distribution is uncertain. Wasserstein distributionally robust optimization (DRO) addresses this issue by optimizing for the worst case distribution within an ambiguity set surrounding the observed data [4]. In this project, I implement a formulation of Wasserstein DRO as a 1-dimensional linear regression problem, and compare its performance to sample average approximation (SAA) in the form of least absolute deviation (LAD) regression [2].

1 Problem & Motivation

Modeling techniques based on SAA assume that the observed data is representative of the underlying distribution and is sampled i.i.d. In practice, however, such assumptions may not hold. In such instances, a model reliant on minimizing only empirical error may achieve satisfactory in-sample performance, but poor out-of-sample performance [1]. Confronted with this situation, one may wish to minimize the expected value of a loss function over a range of possible distributions. Wasserstein DRO accomplishes this by encapsulating the empirical distribution in a ball (an ambiguity set), containing all distributions within Wasserstein radius ϵ [4]. This project explores this idea by solving a 1-dimensional linear regression problem using Wasserstein DRO, as derived by Chen and Paschalidis [2], and then comparing it to a non-robust SAA method, taking the form of standard LAD regression [2].

2 Literature Review

To motivate the use of distributionally robust optimization in regression, I first cover the technique of sample average approximation (SAA). SAA is a technique employed in data-driven optimization which is often tractable, and performs well when sample sizes are high. Mechanically, SAA approximates the distribution by averaging over the observations [1]. This method relies on the assumptions that the observed samples are independent and identically distributed, as well as being representative of the underlying distribution.

Though, its limitations become more apparent when sample size is small, or perhaps not representative of the true distribution. SAA takes the general form

$$\min_{x \in X} \frac{1}{N} \sum_{j=1}^N c(x; \xi^j)$$

Where N is sample size, and $c(x; \xi^j)$ is a cost function, with ξ a random vector [1]. In this project, the method of LAD regression, a form of SAA, is studied as a closely related reference to the distributionally robust implementation. LAD problems take the form

$$\inf_{\beta \in \mathcal{B}} \frac{1}{N} \sum_{i=1}^N |y_i - x_i^\top \beta|$$

[2]. In situations where the data-generating distribution is uncertain, a more robust solution is warranted.

Distributionally Robust Optimization (DRO) addresses this issue by minimizing the expected value of a loss function over all possible distributions in an *ambiguity set*. Mathematically, DRO problems generally take the form

$$\inf_{x \in \mathcal{X}} \sup_{\mathbb{P} \in \mathcal{P}} \mathbb{E}[\ell(x, Z)],$$

where ℓ is a loss function, Z is a random vector drawn from $\mathbb{P}$, and $\mathcal{P}$ is an ambiguity set of possible distributions [3]. The Wasserstein distance is one such way to construct ambiguity sets. The Wasserstein distance is a method of attributing a notion of distance between probability distributions, and can be thought of as the optimal cost of transporting the mass of one distribution to another [5]. In a metric space, this cost is given by a ground metric, taking the form of a p -norm such as $\|\cdot\|_2$ or $\|\cdot\|_1$. For notational clarity, the presentation from [5] is shown. Formally, the Wasserstein distance can be defined as

$$W_p(\mu_1, \mu_2) = \inf_{\gamma \in \Gamma(\mu_1, \mu_2)} \left(\int_{X \times X} d^p(x_1, x_2) d\gamma(x_1, x_2) \right)^{1/p}$$

[5].

In this general definition, μ_1 and μ_2 are probability measures living in a metric space X , which for the scope of this project, can be interchanged with probability distributions. $\Gamma(\mu_1, \mu_2)$ is the set of all *transportation plans* between distributions μ_1 and μ_2 , which defines how mass is sent from one distribution to the next. The integral can be interpreted as the total cost of transportation under the plan γ ; i.e., the sum of the cost (p -norm) of transportation weighted by the mass being transported [5].

The Wasserstein ball serves as an ambiguity set, which contains all distributions within ϵ of the observed data, denoted by ν . This is formally defined as

$$B_\epsilon(\nu) = \{\mu \in \mathcal{P}(X) \mid W_p(\mu, \nu) \leq \epsilon\}$$

[5]. Setting ϵ to 0 reduces the problem to SAA, where the ambiguity set contains only the reference distribution [4]. This connection opens up a natural comparison of Wasserstein regression to SAA-based regression under distributional uncertainty, which is explored in this project.

A challenge of the Wasserstein DRO approach is that, in general, the worst-case expectation $\sup_{\mathbb{P} \in \mathcal{P}} \mathbb{E}[\ell(x, Z)]$ is an infinite-dimensional problem. However, Esfahani and Kuhn [4] show that when the loss function can be expressed as a pointwise maximum of concave functions, and that the Wasserstein ambiguity set is centered around finitely many data points, the problem can be reformulated into a finite-dimensional convex optimization problem. Of particular relevance to the linear regression problem being studied in this project (which uses LAD as a loss function), Esfahani and Kuhn [4] also show that worst-case expectation problems governed by piecewise affine loss functions can be solved as a conic program.

Chen and Paschalidis (2018) [2] further extend these results into regression problems, of which LADs are the focus of this project. The loss function being minimized is the worst-case absolute residual over the ambiguity set, taking the form

$$\inf_{\beta \in \mathcal{B}} \sup_{\mathbb{P} \in \mathcal{P}} |y - x^\top \beta|,$$

where β is the regression coefficient, and $\mathcal{P}$ is the Wasserstein ambiguity set. This general form can be expressed as the problem

$$\inf_{\beta \in \mathcal{B}} \epsilon \|(-\beta, 1)\|_* + \frac{1}{N} \sum_{i=1}^N |y_i - x_i^\top \beta|$$

[2]. Where ϵ is the Wasserstein radius, and β is the regression coefficient. This formulation is a kind of regularized LAD, reminiscent of LASSO and ridge regression. Varying ϵ controls the size of the Wasserstein ambiguity set being robustified over, and consequently, how conservative the model becomes [2]. This is explored in further detail as a simulation in this project.

3 Methods

The goal of the simulation is to implement Wasserstein DRO as a linear regression problem. The loss function being studied is *least absolute deviation* (LAD), rather than least squares. The W_2 Wasserstein metric is chosen, as it is readily implementable as a second-order conic program (SOCP). Following the reformulation in Chen and Paschalidis [2], this project implements:

$$\inf_{\beta} \frac{1}{N} \sum_{i=1}^N |y_i - \mathbf{x}_i^\top \beta| + \epsilon \|(-\beta, 1)\|_2$$

Because the W_2 metric is defined using ℓ_2 , which is self-dual, the LAD is regularized by $\|\cdot\|_2$. $\beta = (m, b)$ is a vector containing the regression coefficients, ϵ is the Wasserstein radius, y_i is the i -th response, and x_i

is the i -th input. For the one-dimensional linear regression problem, the scalar form is:

$$\inf_{\beta} \frac{1}{N} \sum_{i=1}^N |y_i - (mx_i + b)| + \epsilon \sqrt{m^2 + b^2 + 1},$$

where m is the slope of the regression line, and b is the intercept. The robust linear regression problem then takes the form of the following SOCP:

$$\begin{aligned} \min_{a, b_1, \dots, b_N, \beta} \quad & a\epsilon + \frac{1}{N} \sum_{i=1}^N b_i \\ \text{s.t.} \quad & \|(-m, -b, 1)\|_2 \leq a \\ & y_i - mx_i - b \leq b_i, i = 1, \dots, N \\ & -(y_i - mx_i - b) \leq b_i, i = 1, \dots, N \\ & b_i \geq 0, i = 1, \dots, N \end{aligned}$$

[2]. This problem is solved in Julia using JuMP, and the chosen solver is SCS (splitting conic solver).

A standard, unregularized LAD formulation, (a form of SAA) is used as a reference point for the assumption that the sample distribution is representative.

$$\inf_{m, b} \frac{1}{N} \sum_{i=1}^N |y_i - mx_i - b|$$

Model performance is evaluated using the *mean absolute error* (MAE), defined as

$$\frac{1}{N} \sum_{i=1}^N |y_i - (m^* x_i + b^*)|,$$

where m^* and b^* denote the fitted regression parameters. This method of error measurement was chosen in order to remain consistent with the LAD formulation.

The simulations are conducted using synthetically generated one-dimensional data following a known line

$$y_i = 2x_i - 1 + \eta_i,$$

where η_i represents noise. The input points x_i are sampled uniformly over a fixed interval $[-1, 1]$, and noise is added to each response.

4 Simulations

4.1 1-D regression lines against synthetic data

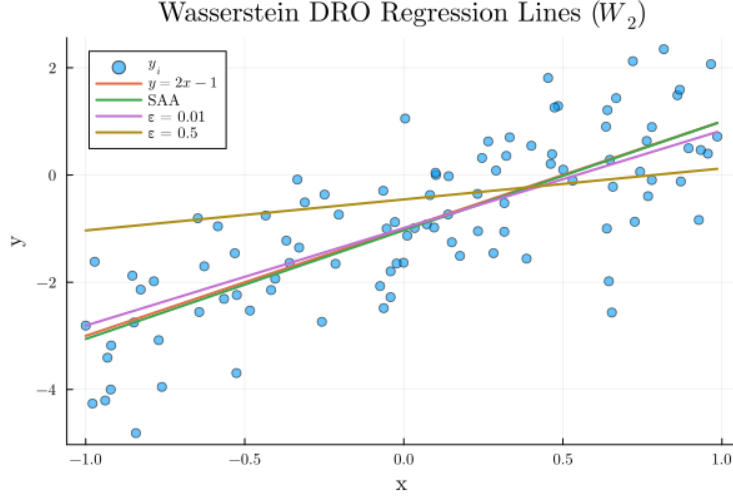


Figure 1: Robust regression lines for various Wasserstein radii, LAD, and the analytical line.

This figure depicts the resultant regression lines as ϵ is varied. The line generated by the SAA approach, is also depicted. The SAA regression line fits the sample well, and overlaps the analytical line which was used to generate the data. It can be noticed that the act of increasing the Wasserstein radius has the effect of worsening the regression line's fit to the sample data by flattening the line.

This effect is observed due to the presence of the regression coefficient in the regularizer term. As ϵ increases, the regression coefficient is further penalized. This, in effect, modulates the level of ambiguity in the distribution that the model robustifies against [2]. The in-sample error is considered more closely in the following figures.

4.2 In-sample Performance for varying ϵ

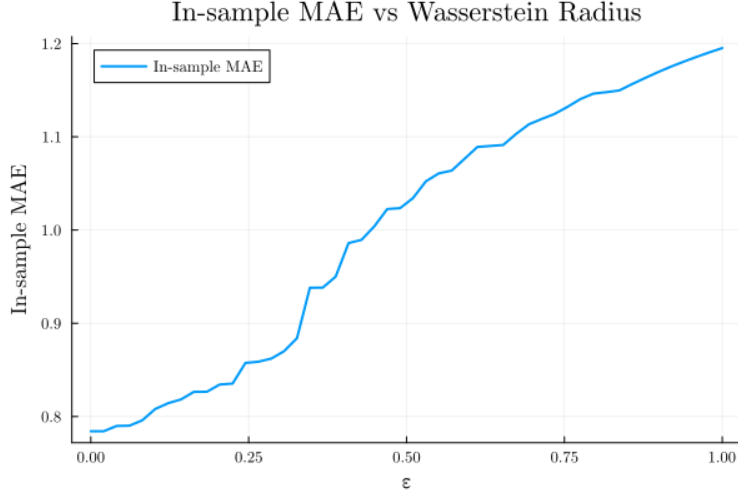


Figure 2: In-sample error tends to increase as the Wasserstein radius ϵ increases

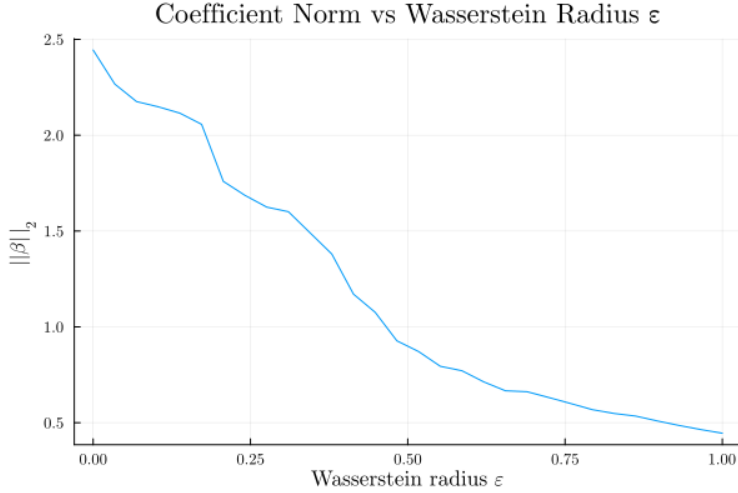


Figure 3: Norm of regression coefficient vector β decreases as ϵ grows.

To further illustrate the impact of varying the Wasserstein radius, ϵ is varied and its impact on both model fit and parameter magnitude is observed. Increasing ϵ enlarges the ambiguity set, which leads the robust model to arrive at a more conservative estimate. This effect, as we can observe, comes at the cost of in-sample fit. Figure 3 offers an explanation for this behavior from the perspective of the regression coefficients. We see that the regression coefficients decrease in magnitude as the ambiguity set enlarges, the very coefficients which are used to regularize the implemented formulation. These depictions show how the act of modulating the radius of the Wasserstein ambiguity set translates into a regularization term that modulates the model's conservativeness to distributional uncertainty [2].

4.3 Wasserstein DRO vs. SAA error variance in small sample size

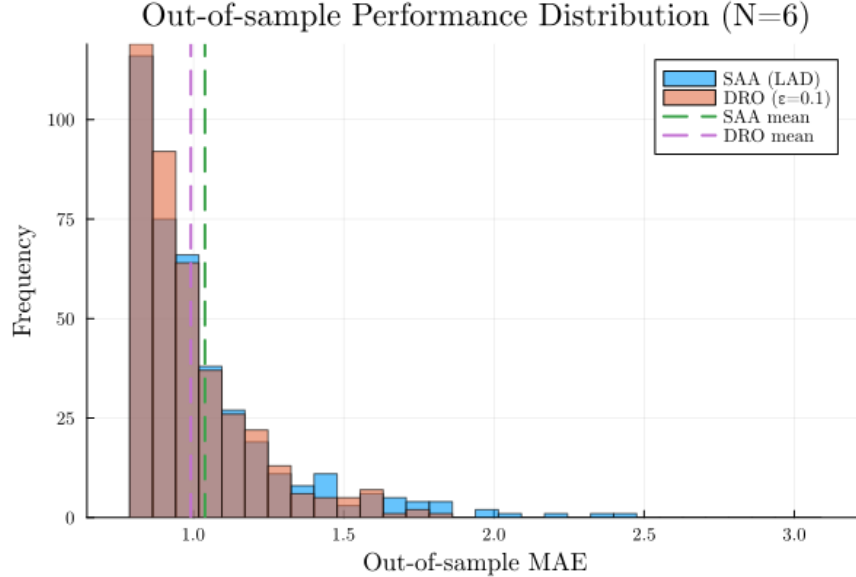


Figure 4: Histogram depicting out-of-sample error frequencies for both SAA and DRO when sample size is very small

This figure demonstrates Wasserstein DRO’s strength of robustness over SAA when the number of observations are very small. In this experiment, a robust LAD and an unregularized LAD are trained on $N = 6$ data points, which creates a high-variance environment. The models are then tested against the true distribution, consisting of 10,000 data points, and MAE is recorded for each model. This experiment is then repeated 400 times.

The results show that, while average out-of-sample MAE for DRO and SAA model are similar, there is a reduction in the incidence of extremely poor fits in the robust model. Specifically, the worst MAE recording for the robust model occurs at ≈ 1.85 , whereas the SAA-based model has MAE’s going out to 2.5. The standard deviation of the SAA model’s MAE in the test set is 0.28, whereas the robust model has a smaller standard deviation of 0.19. Additionally, in terms of raw counts, the worst 10% of SAA MAE on the test data is 1.18, whereas the same figure for the robust case is smaller, at 1.10.

5 Conclusion

This project explored how linear regression using Wasserstein DRO behaves in relation to ordinary LAD regression when the data-generating distribution is uncertain. Based on these simulations and a review of the literature, it can be seen that the tractable formulation’s regularization qualities coincide with the geometric intuition of Wasserstein ambiguity sets. Namely, we see how enlarging the Wasserstein radius makes for a more conservative estimate, at the cost of in-sample performance.

The most interesting of simulations in this project was the histogram depicting the frequency of different error levels attained by the regularized LAD and the standard LAD when sample sizes are extremely small. One could clearly observe how much smaller the variance in the DRO error was, compared to SAA.

I learned a great deal from this project. I am very interested in systems involving uncertainty, so it was a treat to be able to explore optimization through that lens. One of my favorite takeaways from this project is the simplicity of the resultant tractable formulation of Wasserstein DRO. It begins as an abstract, deeply measure-theoretic concept, and gradually becomes a very elegant and implementable formulation that retains most of the geometric properties of its original form.

With more time, I would like to explore Wasserstein regression applied to high dimensional problems, as the way in which features interact with each other can make for some very rich results that cannot be observed in the 1-dimensional case. Additionally, I would like to go further into the dual reformulation.

References

- [1] Dimitris Bertsimas, Vishal Gupta, and Nathan Kallus. Robust sample average approximation. *Mathematical Programming*, 171(1–2):217–282, 2018.
- [2] Ruidi Chen and Ioannis Ch. Paschalidis. A robust learning approach for regression models based on distributionally robust optimization. *Journal of Machine Learning Research*, 19(1):517–564, 2018.
- [3] Daniel Kuhn, Saeed Shafiee, and Wolfram Wiesemann. Distributionally robust optimization. *Acta Numerica*, 34:579–804, 2025.
- [4] Peyman Mohajerin Esfahani and Daniel Kuhn. Data-driven distributionally robust optimization using the wasserstein metric: performance guarantees and tractable reformulations. *Mathematical Programming*, 171(1–2):115–166, 2018.
- [5] Min-Cheng Yue, Daniel Kuhn, and Wolfram Wiesemann. On linear optimization over wasserstein balls. *Mathematical Programming*, 195(1–2):1107–1122, 2022.

A Julia code for simulations

```
using JuMP
using SCS
using LinearAlgebra
using Plots
```



```

using LaTeXStrings

# generate synthetic data about a known line
function sample_data(n)
    x = 2 .* rand(n) .- 1 # random feature

    m_true = 2.0 # slope
    b_true = -1.0 # intercept

    sigma = 1.0 # standard dev
    y = m_true .* x .+ b_true .+ sigma .* randn(n) # randomized y value

    return x, y, m_true, b_true
end

# store the sample data, 100 points
x, y, m_true, b_true = sample_data(100)

# W2 Wasserstein SOCP (Euclidean transport cost)
function solve_W2(x, y, epsilon)
    n = length(x)

    model = Model(SCS.Optimizer)
    set_silent(model)

    # define variables
    @variable(model, m)
    @variable(model, b)
    @variable(model, s >= 0)
    @variable(model, t[1:n] >= 0)

    # LAD loss function
    @constraint(model, [i=1:n], y[i] - (m*x[i] + b) <= t[i])
    @constraint(model, [i=1:n], -(y[i] - (m*x[i] + b)) <= t[i])

    # SOC constraint
    @constraint(model, [s; m; b; 1] in SecondOrderCone())

    # objective

```

```

@objective(model, Min, sum(t)/n + epsilon * s)

optimize!(model)

return value(m), value(b) # return slope and intercept
end

# create a function to calculate the mean absolute error
function mae(x, y, m, b)
    mean(abs.(y .- (m .* x .+ b)))
end

# test some epsilons
1 = 0.0 # reduces to sample average approximation
2 = 0.05
3 = 0.6

# return coefficients from solving the dro problem
m1, b1 = solve_W2(x, y, 1)
m2, b2 = solve_W2(x, y, 2)
m3, b3 = solve_W2(x, y, 3)

# plot results
x_line = range(minimum(x), maximum(x), length=200)

p = plot( x, y, seriestype=:scatter, label=L"y_i",
    title=L"Wasserstein DRO Regression Lines ($W_2$)", xlabel="x", ylabel="y",
    fontfamily="Computer Modern")

plot!(p, x_line, m_true .* x_line .+ b_true, lw=2, label=L"y=2x-1")
plot!(p, x_line, m1 .* x_line .+ b1, lw=2, label="SAA")
plot!(p, x_line, m2 .* x_line .+ b2, lw=2, label=L"$\epsilon$ = 0.01")
plot!(p, x_line, m3 .* x_line .+ b3, lw=2, label=L"$\epsilon$ = 0.5")

epsilons = range(0, 1, length=50) # generate epsilons
mae_vals = Float64[] # container for error values

# test each epsilon

```

```

for epsilon in epsilons
    m, b = solve_W2(x,y,epsilon) # solve the problem for new radius
    error = mean(abs.(y .- (m .* x .+ b))) # calculate the MAE
    push!(mae_vals, error)
end

# plot results
p3 = plot(epsilons, mae_vals, xlabel=L"$\epsilon$", ylabel="In-sample MAE",
    title="In-sample MAE vs Wasserstein Radius", label="In-sample MAE", lw=2,
    fontfamily="Computer Modern")

display(p3)

# epsilons to test
epsilon_vals = range(0.0, 1.0, length=30)

# container for norms
norms = Float64[]

# loop over each epsilon
for epsilon in epsilon_vals
    m_hat, b_hat = solve_W2(x,y,epsilon) # return regression coefficients
    push!(norms, norm([m_hat, b_hat]))
end

# plot results
plot(epsilon_vals, norms, xlabel=L"Wasserstein radius $\epsilon$",
    ylabel=L"$||\beta||_2$", title="Coefficient Norm vs Wasserstein Radius ",
    legend=false, fontfamily="Computer Modern")

# compare DRO to SAA performance when training data is limited

trials = 400
test_n = 10000
epsilon = 0.1
train_n = 6

# collect errors
mae_dro = Float64[]

```

```

mae_saa = Float64[]

# loop through each trial
for _ in 1:trials

    # train test split
    x_train, y_train, _, _ = sample_data(train_n)
    x_test, y_test, _, _ = sample_data(test_n)

    # get slope and intercept for each model
    m_saa, b_saa = solve_W2(x_train, y_train, 0.0)
    m_dro, b_dro = solve_W2(x_train, y_train, epsilon)

    # calculate and record errors
    push!(mae_saa, mae(x_test, y_test, m_saa, b_saa))
    push!(mae_dro, mae(x_test, y_test, m_dro, b_dro))
end

# set up histogram

# define edges
edges = range(min(minimum(mae_saa), minimum(mae_dro)), max(maximum(mae_saa),
    maximum(mae_dro)), length=31)

p5 = histogram(mae_saa, bins=edges, label="SAA (LAD)",
    xlabel="Out-of-sample MAE", ylabel="Frequency",
    title="Out-of-sample Performance Distribution (N=6)", fontfamily="Computer Modern", alpha=0.5)

# DRO histogram
histogram!(p5, mae_dro, bins=edges, label=L"DRO  $\epsilon=0.1$ ", alpha=0.7)

# set up vlines for the mean MAEs across all tests
vline!(p5, [mean(mae_saa)], label="SAA mean", linestyle=:dash)
vline!(p5, [mean(mae_dro)], label="DRO mean", linestyle=:dash)

# print moment figures
println("SAA mean MAE: ", mean(mae_saa), " std: ", std(mae_saa))
println("DRO mean MAE: ", mean(mae_dro), " std: ", std(mae_dro))

```

```
# last 10%
println("SAA worst 10% mean: ", mean(sort(mae_saa)[360:end]))
println("DRO worst 10% mean: ", mean(sort(mae_dro)[360:end]))

display(p5)
```